

1. Lexical Analyzer

lexical_analyzer.c

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int isKeyword(char str[]) {
    char keywords[][10] = {
        "int", "float", "if", "else", "while",
        "for", "return", "char", "void"
    };
};
int n = sizeof(keywords) / sizeof(keywords[0]);
for (int i = 0; i < n; i++) {
    if (strcmp(str, keywords[i]) == 0)
        return 1;
}
return 0;
}
int main() {
    char filePath[200];
    FILE *fp;
    char ch;
    printf("Enter the file path: ");
    scanf("%199s", filePath);
    fp = fopen(filePath, "r");
    if (fp == NULL) {
        printf("Error: Unable to open file.\n");
        return 1;
    }
    while ((ch = fgetc(fp)) != EOF) {
        if (isspace(ch))
            continue;
        if (isalpha(ch) || ch == '_') {
            char token[100];
            int i = 0;
            do {
                token[i++] = ch;
                ch = fgetc(fp);
            } while (isalnum(ch) || ch == '_');
            token[i] = '\0';
            if (isKeyword(token))
                printf("%s : Keyword\n", token);
            else
                printf("%s : Identifier\n", token);
        }
```

```
        if (ch != EOF)
            ungetc(ch, fp);
    }
    else if (isdigit(ch)) {
        char token[100];
        int i = 0;
        do {
            token[i++] = ch;
            ch = fgetc(fp);
        } while (isdigit(ch));
        token[i] = '\0';
        printf("%s : Number\n", token);
        if (ch != EOF)
            ungetc(ch, fp);
    }
    else if (ch == '/') {
        char next = fgetc(fp);
        if (next == '*') {
            while ((ch = fgetc(fp)) != EOF) {
                if (ch == '*') {
                    ch = fgetc(fp);
                    if (ch == '/')
                        break;
                }
            }
        }
        else if (next == '/') {
            while ((ch = fgetc(fp)) != '\n' && ch != EOF);
        }
        else
            printf("/ : Operator\n");
        ungetc(next, fp);
    }
    else if (strchr("+-*/= %<>", ch)) {
        printf("%c : Operator\n", ch);
    }
    else if (strchr("{}[];,:", ch)) {
        printf("%c : Special Symbol\n", ch);
    }
    else {
        printf("%c : Unknown\n", ch);
    }
}
fclose(fp);
return 0;
}
```

Output

input.c

```
int main() {
    int a = 10;
    float b = 20;
    if (a < b)
        return a;
}
```

Enter the file path: input.c

```
int : Keyword
main : Identifier
( : Special Symbol
) : Special Symbol
{ : Special Symbol
int : Keyword
a : Identifier
= : Operator
10 : Number
; : Special Symbol
float : Keyword
b : Identifier
= : Operator
20 : Number
; : Special Symbol
if : Keyword
( : Special Symbol
a : Identifier
< : Operator
b : Identifier
) : Special Symbol
return : Keyword
a : Identifier
; : Special Symbol
} : Special Symbol
```

2. E-Closure

eclosure.c

```
#include <stdio.h>
#define MAX 20
int n, t;
char states[MAX];
```

```
char from[MAX], symbol[MAX], to[MAX];
int visited[MAX];
int getIndex(char ch)
{
    for (int i = 0; i < n; i++)
    {
        if (states[i] == ch)
            return i;
    }
    return -1;
}
void epsilonClosure(char state)
{
    int idx = getIndex(state);
    if (visited[idx])
        return;
    visited[idx] = 1;
    printf("%c ", state);
    for (int i = 0; i < t; i++)
    {
        if (from[i] == state && symbol[i] == 'e')
        {
            epsilonClosure(to[i]);
        }
    }
}
int main()
{
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter the states:\n");
    for (int i = 0; i < n; i++)
    {
        scanf(" %c", &states[i]);
    }
    printf("Enter number of transitions: ");
    scanf("%d", &t);
    printf("Enter transitions (From Symbol To)\n");
    printf("Use 'e' for epsilon.\n");
    for (int i = 0; i < t; i++)
    {
        scanf(" %c %c %c", &from[i], &symbol[i], &to[i]);
    }
    printf("\nEpsilon Closures:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
```

```

        visited[j] = 0;
        printf("E-Closure(%c) = { ", states[i]);
        epsilonClosure(states[i]);
        printf("}\n");
    }
    return 0;
}

```

Output

```

Enter number of states: 5
Enter the states:
0 1 2 3 4
Enter number of transitions: 8
Enter transitions (From Symbol To)
Use 'e' for epsilon.
0 a 1
0 e 1
1 e 2
2 b 3
2 e 4
3 a 4
4 e 3
2 a 4

```

```

Epsilon Closures:
E-Closure(0) = { 0 1 2 4 3 }
E-Closure(1) = { 1 2 4 3 }
E-Closure(2) = { 2 4 3 }
E-Closure(3) = { 3 }
E-Closure(4) = { 4 3 }

```

3. Operator Precedence Parser

op_precedence_parser.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 200
const char symbols[] = { 'i', '+', '-', '*', '/', '^', '(',
    ')', '$' };
const char *symbol_names[] = { "id", "+", "-", "*", "/", "^", "(", ")", "$"
};
#define NSYM 9

```

```

const char table[NSYM][NSYM] = {
    /* id */ { '<', '>', '>', '>', '>', '>', '>', '>', '>' },
    /* + */ { '<', '>', '>', '<', '<', '<', '<', '<', '>' },
    /* - */ { '<', '>', '>', '<', '<', '<', '<', '<', '>' },
    /* * */ { '<', '>', '>', '>', '>', '<', '<', '<', '>' },
    /* / */ { '<', '>', '>', '>', '>', '<', '<', '<', '>' },
    /* ^ */ { '<', '>', '>', '>', '>', '<', '<', '<', '>' },
    /* ( */ { '<', '<', '<', '<', '<', '<', '<', '<', '=' },
    /* ) */ { '<', '>', '>', '>', '>', '>', '>', '>', '>' },
    /* $ */ { '<', '<', '<', '<', '<', '<', '<', '<', '<' }
};

int index_of(char c) {
    for (int i = 0; i < NSYM; i++)
        if (symbols[i] == c) return i;
    return -1;
}

char stack[MAX];
int top = -1;
void push(char c) { stack[++top] = c; }
char pop(void) { return stack[top--]; }
char peek(void) { return stack[top]; }

void print_stack(void) {
    for (int i = 0; i <= top; i++) {
        if (stack[i] == 'i') printf("id");
        else printf("%c", stack[i]);
    }
}

int tokenize(const char *expr, char *tokens) {
    int n = 0, len = strlen(expr);
    for (int i = 0; i < len; i++) {
        char c = expr[i];
        if (isspace((unsigned char)c)) continue;
        if (isalnum((unsigned char)c)) {
            tokens[n++] = 'i';
            while (i + 1 < len && isalnum((unsigned char)expr[i +
1])) i++;
        } else if (strchr("+-*/^()", c)) {
            tokens[n++] = c;
        } else {
            printf("Unrecognized character '%c' in input.\n", c);
            exit(1);
        }
    }
    tokens[n++] = '$';
    return n;
}

```

```

int parse(const char *tokens, int ntok) {
    int ip = 0;
    top = -1;
    push('$');
    printf("\n%-20s %-20s %-10s\n", "STACK", "INPUT", "ACTION");

    printf("-----\n");

    while (1) {
        char a = peek();
        char b = tokens[ip];
        printf("%-20.*s", 0, "");
        {
            char sbuf[MAX] = {0};
            int p = 0;
            for (int i = 0; i <= top; i++) {
                if (stack[i] == 'i') { sbuf[p++]='i'; sbuf[p+
+]='d'; }

                else sbuf[p++] = stack[i];
            }
            sbuf[p] = '\0';
            char ibuf[MAX] = {0};
            int q = 0;
            for (int i = ip; i < ntok; i++) {
                if (tokens[i] == 'i') { ibuf[q++]='i'; ibuf[q+
+]='d'; }

                else ibuf[q++] = tokens[i];
            }
            ibuf[q] = '\0';
            printf("%-20s %-20s", sbuf, ibuf);
        }
        if (a == '$' && b == '$') {
            printf("%-10s\n", "Accept");
            return 1;
        }
        int ra = index_of(a);
        int rb = index_of(b);
        if (ra == -1 || rb == -1) {
            printf("%-10s\n", "Error");
            return 0;
        }
        char rel = table[ra][rb];
        if (rel == '<' || rel == '=') {
            printf("%-10s\n", "Shift");
            push(b);
            ip++;
        }
    }
}

```

```

    } else if (rel == '>') {
        printf("%-10s\n", "Reduce");
        char popped;

        do {
            popped = pop();
            if (top < 0) { printf("Error: stack underflow.
\n"); return 0; }
        } while (table[index_of(peek())][index_of(popped)] !=
'<');
    } else {
        printf("%-10s\n", "Error");
        return 0;
    }
}

int main(void) {
    char expr[MAX];
    char tokens[MAX];
    printf("Operator Precedence Parser\n");
    printf("Grammar operators supported: + - * / ^ ( ) (operands
= id)\n");
    printf("Enter an expression (e.g. id+id*(id-id) or a+b*(c-d)
):\n> ");
    if (!fgets(expr, sizeof(expr), stdin)) return 0;
    expr[strcspn(expr, "\n")] = '\0';
    int ntok = tokenize(expr, tokens);
    int accepted = parse(tokens, ntok);
    printf("\nResult: %s\n", accepted ? "Expression is VALID
(accepted)." : "Expression is INVALID
(rejected).");
    return 0;
}

```

Output

Operator Precedence Parser
Grammar operators supported: + - * / ^ () (operands = id)
Enter an expression (e.g. id+id*(id-id) or a+b*(c-d)):
> a + (b * c)

STACK	INPUT	ACTION	

	\$	id+(id*id)\$	Shift
	\$id	+(id*id)\$	
Reduce			

	\$	+(id*id)\$	Shift
	\$+	(id*id)\$	Shift
	\$+(	id*id)\$	Shift
Reduce	\$+(id	*id)\$	
	\$+(	*id)\$	Shift
	\$+(*	id)\$	Shift
Reduce	\$+(*id	)\$	
Reduce	\$+(*	)\$	
	\$+(	)\$	Shift
Reduce	\$+()	\$	
Reduce	\$+	\$	
Reduce	\$	\$	
Accept			

Result: Expression is VALID (accepted).

4. Recursive Decent Parser

recursive_decent_parser.c

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
char input[100];
int pos = 0;
void skipSpaces() {
    while (isspace(input[pos]))
        pos++;
}
void factor() {
    skipSpaces();
    if (isalpha(input[pos])) {
        pos++;
        while (isalnum(input[pos]))
            pos++;
    } else if (isdigit(input[pos])) {
        while (isdigit(input[pos]))
            pos++;
    } else if (input[pos] == '(') {
        pos++;
        expression();
    }
}
```

```
skipSpaces();
if (input[pos] == ')') {
    pos++;
} else {
    printf("Invalid expression: missing ')\n");
    exit(1);
}
} else {
    printf("Invalid expression\n");
    exit(1);
}
skipSpaces();
}
void term() {
    factor();
    skipSpaces();
    while (input[pos] == '*') {
        pos++;
        factor();
        skipSpaces();
    }
}
void expression() {
    term();
    skipSpaces();
    while (input[pos] == '+') {
        pos++;
        term();
        skipSpaces();
    }
}
int main() {
    printf("Enter an expression: ");
    fgets(input, sizeof(input), stdin);
    expression();
    skipSpaces();
    if (input[pos] == '\0')
        printf("Valid expression\n");
    else
        printf("Invalid expression\n");
    return 0;
}
```

Output

```
Enter an expression: a + (b * c)
Valid expression
```

5. Shift Reduce Parser

sr_parser.c

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
char input[MAX];
int ip = 0;
void push(char c) {
    stack[++top] = c;
}
char pop(void) {
    if (top < 0) return '\0';
    return stack[top--];
}
void print_config(const char *action) {
    int i;
    printf("%-25s", action);
    for (i = 0; i <= top; i++) putchar(stack[i]);
    printf("\t\t\t");
    printf("%s\n", input + ip);
}
int reduce(void) {
    if (top >= 0 && stack[top] == 'i') {
        pop();
        push('E');
        print_config("Reduce: id -> E");
        return 1;
    }
    if (top >= 2 && stack[top] == 'E' && stack[top-1] == '*' &&
        stack[top-2] == 'E') {
        pop(); pop(); pop();
        push('E');
        print_config("Reduce: E*E -> E");
        return 1;
    }
    if (top >= 2 && stack[top] == 'E' && stack[top-1] == '+' &&
        stack[top-2] == 'E') {
        pop(); pop(); pop();
        push('E');
        print_config("Reduce: E+E -> E");
        return 1;
    }
}
```

```
    if (top >= 2 && stack[top] == ')' && stack[top-1] == 'E' &&
        stack[top-2] == '(') {
        pop(); pop(); pop();
        push('E');
        print_config("Reduce: (E) -> E");
        return 1;
    }
    return 0;
}
int main(void) {
    printf("Shift-Reduce Parser\n");
    printf("Enter input string (e.g. i+i*i or (i+i)*i): ");
    if (fgets(input, MAX, stdin) == NULL) {
        printf("No input read.\n");
        return 1;
    }
    input[strcspn(input, "\n")] = '\0';
    printf("\n%-25s%-12s%s\n", "ACTION", "STACK", "INPUT");

    printf("-----\n");
    int len = (int)strlen(input);
    while (1) {
        char next = (ip < len) ? input[ip] : '\0';
        if (top >= 2 && stack[top] == 'E' && stack[top-1] == '+'
            && stack[top-2] == 'E' && next == '*') {
            else if (reduce()) {
                continue;
            }
            if (ip >= len) {
                break;
            }
            char c = input[ip++];
            if (c == 'i' || c == '+' || c == '*' || c == '(' || c ==
                ')') {
                push(c);
                print_config("Shift");
            } else {
                printf("\nError: invalid symbol '%c' in input.\n", c);
                return 1;
            }
        }
        if (top == 0 && stack[0] == 'E' && ip >= len) {
            printf("\nResult: Input string is ACCEPTED by the grammar.\n");
        } else {
            printf("\nResult: Input string is REJECTED (stack = ");

```

```

    for (int i = 0; i <= top; i++) putchar(stack[i]);
    printf(").\n");
}
return 0;
}

```

Output

Shift-Reduce Parser

Enter input string (e.g. i+i*i or (i+i)*i): i+(i*i)

ACTION	STACK	INPUT
Shift	i	+(i*i)
Reduce: id -> E	E	+(i*i)
Shift	E+	(i*i)
Shift	E+(	i*i)
Shift	E+(i	*i)
Reduce: id -> E	E+(E	*i)
Shift	E+(E*	i)
Shift	E+(E*i	)
Reduce: id -> E	E+(E*E	)
Reduce: E*E -> E	E+(E	)
Shift	E+(E)	
Reduce: (E) -> E	E+E	
Reduce: E+E -> E	E	

Result: Input string is ACCEPTED by the grammar.

6. Constant Propagation

constant_propagation.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
struct Statement {
    char lhs[20];
    char op1[20];
    char op2[20];
    char op;
};

```

```

int isConstant(char *str) {
    int i = 0;
    if (str[0] == '-')
        i = 1;
    for (; str[i] != '\0'; i++) {
        if (str[i] < '0' || str[i] > '9')
            return 0;
    }
    return 1;
}

int main() {
    int n, i, value;
    struct Statement s[20];
    printf("Enter number of statements: ");
    scanf("%d", &n);
    printf("Enter statements in the form: x = y + z\n");
    printf("For constant assignment use: x = 10\n\n");
    for (i = 0; i < n; i++) {
        char input[50];
        printf("Statement %d: ", i + 1);
        scanf("%[^\n]", input);
        char *eq = strchr(input, '=');
        if (eq == NULL) {
            printf("Invalid statement!\n");
            i--;
            continue;
        }
        *eq = '\0';
        strcpy(s[i].lhs, input);
        char rhs[30];
        strcpy(rhs, eq + 1);
        char *p = strchr(rhs, '+');
        s[i].op = '+';
        if (!p) {
            p = strchr(rhs, '-');
            s[i].op = '-';
        }
        if (!p) {
            p = strchr(rhs, '*');
            s[i].op = '*';
        }
        if (!p) {
            p = strchr(rhs, '/');
            s[i].op = '/';
        }
    }
}

```

```

if (p) {
    *p = '\0';
    strcpy(s[i].op1, rhs);
    strcpy(s[i].op2, p + 1);
} else {
    strcpy(s[i].op1, rhs);
    strcpy(s[i].op2, "");
    s[i].op = '=';
}

for (int j = 0; s[i].lhs[j]; j++)
    if (s[i].lhs[j] == ' ')
        s[i].lhs[j] = '\0';
for (int j = 0; s[i].op1[j]; j++)
    if (s[i].op1[j] == ' ') {
        memmove(&s[i].op1[j], &s[i].op1[j + 1],
                strlen(&s[i].op1[j]));
        j--;
    }
for (int j = 0; s[i].op2[j]; j++)
    if (s[i].op2[j] == ' ') {
        memmove(&s[i].op2[j], &s[i].op2[j + 1],
                strlen(&s[i].op2[j]));
        j--;
    }
}

printf("\nAfter Constant Propagation:\n");
for (i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        if (strcmp(s[i].op1, s[j].lhs) == 0 &&
            isConstant(s[j].op1)) {
            strcpy(s[i].op1, s[j].op1);
        }
        if (strcmp(s[i].op2, s[j].lhs) == 0 &&
            isConstant(s[j].op1)) {
            strcpy(s[i].op2, s[j].op1);
        }
    }
    if (s[i].op == '=') {
        printf("%s = %s\n", s[i].lhs, s[i].op1);
    }

    else if (isConstant(s[i].op1) &&
              isConstant(s[i].op2)) {
        int a = atoi(s[i].op1);
        int b = atoi(s[i].op2);

```

```

switch (s[i].op) {
    case '+': value = a + b; break;
    case '-': value = a - b; break;
    case '*': value = a * b; break;
    case '/':
        if (b == 0) {
            printf("%s = %s %c %s\n",
                    s[i].lhs, s[i].op1,
                    s[i].op, s[i].op2);
            continue;
        }
        value = a / b;
        break;
}
printf("%s = %d\n", s[i].lhs, value);
sprintf(s[i].op1, "%d", value);
}
else {
    printf("%s = %s %c %s\n",
            s[i].lhs,
            s[i].op1,
            s[i].op,
            s[i].op2);
}
}
return 0;
}

```

Output

Enter number of statements: 5
 Enter statements in the form: x = y + z
 For constant assignment use: x = 10
 Statement 1: a = b + c
 Statement 2: c = 10
 Statement 3: b = a + c
 Statement 4: d = c + 10
 Statement 5: e = c + d

After Constant Propagation:

```

a = b + c
c = 10
b = a + 10
d = 20
e = 30

```

7. Intermediate Code Generation

inter_code_gen.c

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char stack[100];
int top = -1;
int tempCount = 1;
void push(char c) {
    stack[++top] = c;
}
char pop() {
    return stack[top--];
}
int precedence(char op) {
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return 0;
}
void infixToPostfix(char infix[], char postfix[]) {
    int i, j = 0;
    char c;
    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];
        if (isdigit(c)) {
            postfix[j++] = c;
        }
        else if (c == '(') {
            push(c);
        }
        else if (c == ')') {
            while (top != -1 && stack[top] != '(')
                postfix[j++] = pop();
            pop();
        }
        else {
            while (top != -1 &&
                precedence(stack[top]) >= precedence(c))
                postfix[j++] = pop();
            push(c);
        }
    }
}
```

```
while (top != -1)
    postfix[j++] = pop();
postfix[j] = '\0';
}
void generateTAC(char postfix[]) {
    char operandStack[100][20];
    int opTop = -1;
    int i;
    char op1[20], op2[20], result[20];
    for (i = 0; postfix[i] != '\0'; i++) {
        if (isdigit(postfix[i])) {
            sprintf(operandStack[++opTop], "%c", postfix[i]);
        }
        else {
            strcpy(op2, operandStack[opTop--]);
            strcpy(op1, operandStack[opTop--]);
            sprintf(result, "t%d", tempCount++);
            printf("%s = %s %c %s\n",
                result, op1, postfix[i], op2);
            strcpy(operandStack[++opTop], result);
        }
    }
}
int main() {
    char expression[100];
    char postfix[100];
    printf("Enter expression: ");
    scanf("%s", expression);
    infixToPostfix(expression, postfix);
    printf("\nThree Address Code:\n");
    generateTAC(postfix);
    return 0;
}
```

Output

Enter expression: x=a+b*c

Three Address Code:

```
t1 = b * c
t2 = a + t1
t3 = x = t2
```

8. Convert Three Address Code to 8086 Assembly Language

assembly_converter.c

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[50];
    char res, arg1, op, arg2;
    printf("Enter the three address code:\n");
    if (scanf("%s", str) != 1) {
        return 1;
    }
    if (sscanf(str, "%c=%c%c%c", &res, &arg1, &op, &arg2) == 4) {
        printf("\n");
        printf("MOV R0,%c\n", arg1);
        switch (op) {
            case '+':
                printf("ADD R0,%c\n", arg2);
                break;
            case '-':
                printf("SUB R0,%c\n", arg2);
                break;
            case '*':
                printf("MUL R0,%c\n", arg2);
                break;
            case '/':
                printf("DIV R0,%c\n", arg2);
                break;
            default:
                printf("; Unknown operator\n");
                return 1;
        }
        printf("MOV %c,R0\n", res);
    } else {
        printf("Invalid three address code format.\n");
    }
    return 0;
}
```

Output

Enter the three address code:
x=a+b

```
MOV R0,a
ADD R0,b
MOV x,R0
```

9. Lexical Analyzer Using Lex Tool

lexer.l

```
%{
#include <stdio.h>
#include <stdlib.h>
int line_number = 1;
}%
DIGIT      [0-9]
LETTER     [a-zA-Z_]
ID         {LETTER}({LETTER}|{DIGIT})*
INT_CONST  {DIGIT}+
FLOAT_CONST {DIGIT}+\.{DIGIT}+([eE][+-]?{DIGIT}+)?

%%
\n          { line_number++; }
[ \t\r]+   { /* Skip whitespace */ }
"//".*      { /* Ignore single-line comments */ }
"/*"([^\*]|\\*+([^\*]/))*\\*+"/*" {
    /* Ignore multi-line comments; count internal newlines */
    for (int i = 0; yytext[i] != '\\0'; i++) {
        if (yytext[i] == '\\n') line_number++;
    }
}
/* Keywords - must appear BEFORE the generic identifier
pattern */
"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"re
turn"|"break"|"continue" {
    printf("[Line %d] KEYWORD      : %s\n", line_number, yytext);
}
/* Literals / Constants */
{FLOAT_CONST} { printf("[Line %d] FLOAT_CONST : %s\n",
line_number, yytext); }
{INT_CONST}   { printf("[Line %d] INT_CONST   : %s\n",
line_number, yytext); }
\"([^\\"\\\\]|\\\\.)*\" { printf("[Line %d] STRING_LIT   : %s\n",
```

```

line_number, yytext); }
/* Identifiers */
{ID} { printf("[Line %d] IDENTIFIER : %s\n",
line_number, yytext); }
/* Multi-character Operators */
"=="|"!="|"<="|">="|"&&"|"||"|"++"|"--" {
printf("[Line %d] RELOP/LOGOP : %s\n", line_number, yytext);
}
/* Single-character Operators and Delimiters */
[+\-*/%=<>] { printf("[Line %d] OPERATOR : %s\n",
line_number, yytext); }
[;,( ){} \[\]] { printf("[Line %d] DELIMITER : %s\n",
line_number, yytext); }
/* Catch-all for unrecognized characters */
. { printf("[Line %d] ERROR (Unknown Token): %s\n",
line_number, yytext); }
%%
int yywrap(void) {
return 1;
}
int main(int argc, char **argv) {
if (argc > 1) {
FILE *file = fopen(argv[1], "r");
if (!file) {
perror("Failed to open file");
return 1;
}
yyin = file;
}
yylex();
return 0;
}

```

Output

input.c

```

int main() {
float radius = 5.5;
float area;
if (radius >= 0.0) {
area = 3.14 * radius;
}
return 0;
}

```

```

[Line 1] KEYWORD : int
[Line 1] IDENTIFIER : main
[Line 1] DELIMITER : (
[Line 1] DELIMITER : )
[Line 1] DELIMITER : {
[Line 2] KEYWORD : float
[Line 2] IDENTIFIER : radius
[Line 2] OPERATOR : =
[Line 2] FLOAT_CONST : 5.5
[Line 2] DELIMITER : ;
[Line 3] KEYWORD : float
[Line 3] IDENTIFIER : area
[Line 3] DELIMITER : ;
[Line 4] KEYWORD : if
[Line 4] DELIMITER : (
[Line 4] IDENTIFIER : radius
[Line 4] RELOP/LOGOP : >=
[Line 4] FLOAT_CONST : 0.0
[Line 4] DELIMITER : )
[Line 4] DELIMITER : {
[Line 5] IDENTIFIER : area
[Line 5] OPERATOR : =
[Line 5] FLOAT_CONST : 3.14
[Line 5] OPERATOR : *
[Line 5] IDENTIFIER : radius
[Line 5] DELIMITER : ;
[Line 6] DELIMITER : }
[Line 7] KEYWORD : return
[Line 7] INT_CONST : 0
[Line 7] DELIMITER : ;
[Line 8] DELIMITER : }

```

10. Validate Arithmetic Expression using Lex and YACC Tool

parser.y

```

%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
int valid = 1;
%}
%token NUMBER ID
%left '+' '-'

```

```

%left '*' '/'
%left '(' ')'

%%
input:
    /* empty */
    | input line
    ;
line:
    '\n'
    | expr '\n' {
        if (valid) {
            printf("Valid arithmetic expression\n\n");
        }
        valid = 1;
    }
    | error '\n' {
        yyerrok;
        valid = 1;
    }
    ;
expr:
    expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | '(' expr ')'
    | NUMBER
    | ID
    ;

%%

void yyerror(const char *s) {
    valid = 0;
    printf("Invalid arithmetic expression\n\n");
}

int main(void) {
    printf("Enter arithmetic expressions (Press Ctrl+D to exit):\n");
    yyparse();
    return 0;
}

```

lexer.l

```

%{
#include "y.tab.c"
#include <stdio.h>
%}

DIGIT [0-9]
LETTER [a-zA-Z_]

%%
[ \t]+ { /* Ignore whitespace */ }
{DIGIT}+(\.{DIGIT}+)? { return NUMBER; }
{LETTER}({LETTER}|{DIGIT})* { return ID; }
[+\-*/()\n] { return yytext[0]; }
. {
    printf("Lexical Error: Unrecognized character '%s'\n",
yytext);
    return yytext[0];
}

%%

int yywrap(void) {
    return 1;
}

```

Output

Enter arithmetic expressions (Press Ctrl+D to exit):

a + b * c

Valid arithmetic expression

a ++ b

Invalid arithmetic expression